

# Module 2: Database Objects

---

Understanding database objects is essential because every SQL operation revolves around these objects. Before writing complex queries, you must know how data is organized, stored, and related inside a database.

---



## Introduction

---

A database is not just a collection of tables.

Modern relational databases contain multiple objects that work together to store, manage, secure, and retrieve data efficiently.

Some of the most commonly used database objects are:

- Databases
- Tables
- Columns
- Rows
- Schemas
- Constraints
- Keys
- Views
- Indexes
- Stored Procedures
- Functions
- Triggers

In this module, we focus on the foundational database objects that every SQL learner must master.

---



## Learning Objectives

---

After completing this module, you will be able to:

- ✓ Understand database architecture
- ✓ Differentiate between databases, tables, rows, and columns
- ✓ Understand schemas and their purpose
- ✓ Identify various keys used in databases
- ✓ Understand relationships between tables
- ✓ Design simple database structures

✓ Apply database design best practices

---

## Table of Contents

---

1. What are Database Objects?
  2. Database
  3. Table
  4. Row (Record)
  5. Column (Field)
  6. Schema
  7. Keys
  8. Relationships
  9. Constraints Overview
  10. Database Design Example
  11. Best Practices
  12. Summary
  13. Practice Questions
- 

## 1 What are Database Objects?

---

Database Objects are structures used to store, organize, manipulate, and retrieve data.

Think of a database as a company building.

| Object      | Real-Life Example   |
|-------------|---------------------|
| Database    | Entire Building     |
| Schema      | Department Floor    |
| Table       | Room                |
| Row         | Individual File     |
| Column      | Information Section |
| Primary Key | File Number         |

---

## Common Database Objects

---

| Object   | Purpose         |
|----------|-----------------|
| Database | Stores all data |
| Table    | Stores records  |
| View     | Virtual table   |

| Object           | Purpose               |
|------------------|-----------------------|
| Index            | Improves performance  |
| Stored Procedure | Reusable SQL code     |
| Function         | Performs calculations |
| Trigger          | Automatic action      |
| Constraint       | Enforces rules        |

## 2 Database

A Database is an organized collection of related data.

### Example

Consider an E-Commerce Company.

Database:

ECommerceDB

Contains:

Customers  
Orders  
Products  
Payments  
Suppliers

All these tables together form a database.

### Visual Representation

ECommerceDB

- Customers
- Orders
- Products
- Payments
- Suppliers

# Creating a Database

## SQL Server Syntax

```
CREATE DATABASE CompanyDB;
```

## Viewing Databases

```
SELECT name  
FROM sys.databases;
```

## Selecting Database

```
USE CompanyDB;
```

## 3 Table

A Table is the most important database object.

It stores data in rows and columns.

## Example Table

### Employees

| EmployeeID | Name  | Salary |
|------------|-------|--------|
| 101        | Kunal | 45000  |
| 102        | Rahul | 50000  |

## Table Structure

Employees

| Column Name | Data Type |
|-------------|-----------|
| EmployeeID  | INT       |

| Column Name | Data Type |
|-------------|-----------|
| Name        | VARCHAR   |
| Salary      | DECIMAL   |

---

## Creating Table

```
CREATE TABLE Employees
(
    EmployeeID INT,
    Name VARCHAR(100),
    Salary DECIMAL(10,2)
);
```

---

## Why Tables Matter

Without tables:

- ✗ Data becomes unorganized
- ✗ Searching becomes difficult
- ✗ Reporting becomes impossible

With tables:

- ✓ Structured storage
  - ✓ Easy querying
  - ✓ Better performance
- 



## Row (Record)

---

A Row represents a single record in a table.

---

## Example

| EmployeeID | Name  | Salary |
|------------|-------|--------|
| 101        | Kunal | 45000  |

This complete line is one row.

---

## Other Names

- Record
  - Tuple
- 

## Example

Employees Table

| EmployeeID | Name  |
|------------|-------|
| 101        | Kunal |
| 102        | Rahul |
| 103        | Aman  |

Number of Rows = 3

---

## 5 Column (Field)

---

A Column represents a specific attribute of data.

---

## Example

| EmployeeID | Name | Salary |
|------------|------|--------|
|------------|------|--------|

Columns:

```
EmployeeID
Name
Salary
```

---

## Other Names

- Field
  - Attribute
- 

## Column Characteristics

Every column has:

Name

Salary

## Data Type

DECIMAL(10,2)

## Constraints

NOT NULL  
UNIQUE  
PRIMARY KEY

---

## Row vs Column

| Feature    | Row          | Column        |
|------------|--------------|---------------|
| Represents | Record       | Attribute     |
| Example    | One Employee | Employee Name |
| Direction  | Horizontal   | Vertical      |

---

## Schema

A Schema is a logical container used to organize database objects.

---

## Why Schema Exists

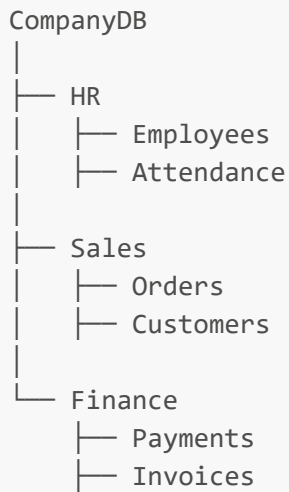
Large organizations may have:

- Hundreds of tables
- Thousands of procedures
- Many developers

Schemas help organize everything.

---

## Example



---

## Default Schema

In SQL Server:

```
dbo
```

stands for:

```
Database Owner
```

---

## Example

```
SELECT *  
FROM dbo.Employees;
```

---

## Creating Schema

```
CREATE SCHEMA HR;
```

---

## Creating Table Under Schema

```
CREATE TABLE HR.Employees  
(
```

```
EmployeeID INT,  
Name VARCHAR(100)  
);
```

## 7 Keys

Keys uniquely identify records and create relationships.

### Primary Key

Uniquely identifies every row.

#### Example

| EmployeeID | Name  |
|------------|-------|
| 101        | Kunal |
| 102        | Rahul |

EmployeeID is Primary Key.

### Rules

Must Be Unique



```
101  
102  
103
```



```
101  
101  
102
```

Cannot Be NULL



NULL

---

## Creating Primary Key

```
CREATE TABLE Employees
(
    EmployeeID INT PRIMARY KEY,
    Name VARCHAR(100)
);
```

---

## Candidate Key

A column that can become a primary key.

---

### Example

| EmployeeID | Email       |
|------------|-------------|
| 101        | a@gmail.com |
| 102        | b@gmail.com |

Both can uniquely identify records.

---

## Alternate Key

Candidate key not chosen as primary key.

Example:

```
Primary Key → EmployeeID
Alternate Key → Email
```

---

## Composite Key

Primary key made using multiple columns.

---

## Example

```
PRIMARY KEY
```

```
(  
  StudentID,  
  CourseID  
)
```

---

## Foreign Key

Used to connect tables.

---

---

## Parent Table

Departments

| DepartmentID | DepartmentName |
|--------------|----------------|
| 1            | HR             |
| 2            | IT             |

---

---

## Child Table

Employees

| EmployeeID | DepartmentID |
|------------|--------------|
| 101        | 1            |
| 102        | 2            |

DepartmentID is Foreign Key.

---

## Creating Foreign Key

```
CREATE TABLE Departments
(
  DepartmentID INT PRIMARY KEY,
  DepartmentName VARCHAR(50)
);
```

```
CREATE TABLE Employees
(
  EmployeeID INT PRIMARY KEY,
  Name VARCHAR(100),

  DepartmentID INT,

  FOREIGN KEY (DepartmentID)
  REFERENCES Departments(DepartmentID)
);
```

---

## 8 Relationships

---

Relationships connect tables.

---

### One-to-One (1:1)

---

One record relates to one record.

---

Example:

Person ↔ Passport

One person has one passport.

---

### One-to-Many (1:M)

---

Most common relationship.

---

Example:

Department → Employees

One department has many employees.

---

Visualization:

```
HR
|
├── Kunal
├── Rahul
└── Aman
```

---

## Many-to-Many (M:M)

---

Many records relate to many records.

---

Example:

Students ↔ Courses

One student can take many courses.

One course can have many students.

---

Solution:

Use Junction Table.

StudentCourses

---

## Example

---

Students

**StudentID**

---

1

## Courses

### **CourseID**

---

10

## StudentCourses

### **StudentID**   **CourseID**

---

1                  10



## Constraints Overview

---

Constraints enforce rules.

---

### NOT NULL

Value required.

Name **VARCHAR(50)** **NOT NULL**

---

### UNIQUE

No duplicates.

Email **VARCHAR(100)** **UNIQUE**

---

### PRIMARY KEY

Unique + Not Null.

---

### FOREIGN KEY

Maintains relationships.

---

### CHECK

Validates data.

```
CHECK (Salary > 0)
```

## DEFAULT

Default value.

```
Country VARCHAR(50)
DEFAULT 'India'
```

# 10 Database Design Example

College Management System

## Students Table

| StudentID | StudentName |
|-----------|-------------|
|-----------|-------------|

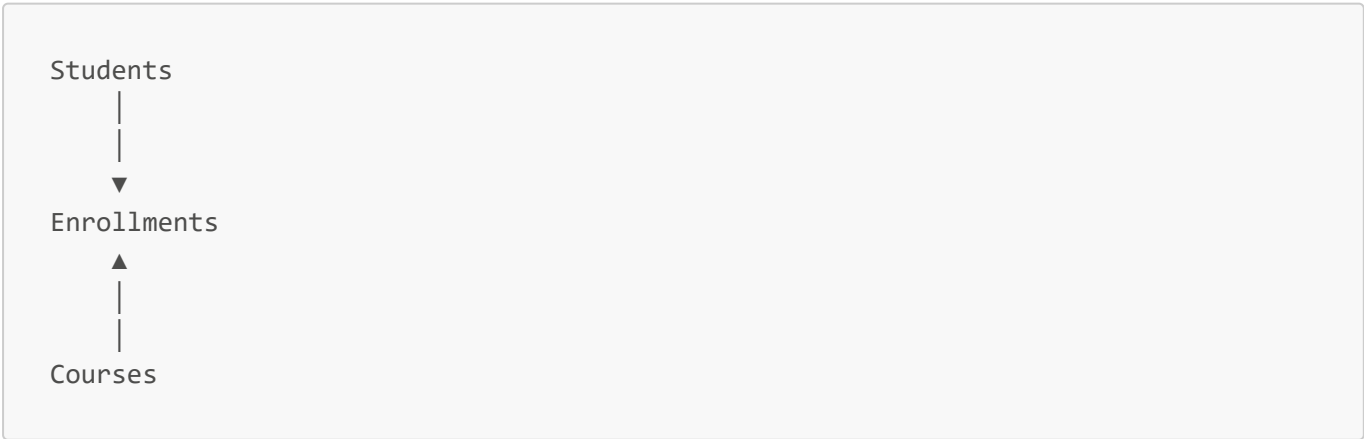
## Courses Table

| CourseID | CourseName |
|----------|------------|
|----------|------------|

## Enrollments Table

| StudentID | CourseID |
|-----------|----------|
|-----------|----------|

## Relationships



# 1 1 Best Practices

---

## Use Meaningful Names



EmployeeID



EID

---

## Define Primary Keys

Every table should have one.

---

## Use Proper Data Types

Salary:

DECIMAL(10,2)

not

VARCHAR(50)

---

## Normalize Data

Avoid duplicate information.

---

## Create Relationships Carefully

Use foreign keys whenever applicable.

---

## Keep Schemas Organized

Separate modules logically.

---



## Summary

---

In this module you learned:

- ✓ Database
  - ✓ Table
  - ✓ Row
  - ✓ Column
  - ✓ Schema
  - ✓ Primary Key
  - ✓ Candidate Key
  - ✓ Alternate Key
  - ✓ Composite Key
  - ✓ Foreign Key
  - ✓ Relationships
  - ✓ Constraints Overview
  - ✓ Database Design Basics
- 



## Practice Questions

---

### Theory

1. What is a database object?
  2. Difference between row and column?
  3. What is a schema?
  4. What is a primary key?
  5. What is a foreign key?
  6. Explain candidate key.
  7. Explain composite key.
  8. What is a one-to-many relationship?
  9. What is a many-to-many relationship?
  10. Why are schemas important?
- 

### Practical

#### Task 1

Create Database:

CollegeDB

---

## Task 2

Create Tables:

Students  
Courses  
Enrollments

---

## Task 3

Add:

- Primary Keys
  - Foreign Keys
- 

## Task 4

Insert Sample Data

---

## Task 5

Draw Relationship Diagram

---



# Next Module

---

→ Module 3: SQL Data Types

Topics Covered:

- Numeric Data Types
- Character Data Types
- Date & Time Data Types
- Boolean Data Types
- Choosing Appropriate Data Types
- Data Type Best Practices